AHFinderDirect – A Fast Apparent Horizon Finder

Jonathan Thornburg <jthorn@aei.mpg.de>

Date: 2009-02-02 20:23:07 -0500 (Mon, 02 Feb 2009)

Abstract

Thorn **AHFinderDirect** locates apparent horizons (or more generally, closed 2-surfaces with S^2 topology having any desired constant expansion) in a numerically computed slice using a direct method, posing the apparent horizon equation as an elliptic PDE on angular-coordinate space. This is very fast and accurate, but requires a "reasonable" initial guess. This thorn guide describes how to use the thorn.

1 Introduction

A "marginally trapped surface" is a closed 2-surface in a slice whose congruence of future-pointing outgoing null geodesics has zero expansion. There may be several such surfaces, some nested inside others; an "apparent horizon" is an outermost marginally trapped surface. In terms of the usual $3+1$ variables, an apparent horizon satisfies the equation

$$\Theta \equiv \nabla_i n^i + K_{ij} n^i n^j - K = 0 \tag{1}$$

where n^i is the outward-pointing unit normal to the apparent horizon, and ∇_i is the covariant derivative operator associated with the 3-metric g_{ij} in the slice. (See [Yor89] for a derivation of equation (1).) (Optionally, you can replace the right hand side of (1) by any specified nonzero constant, i.e. you can find a surface of constant (in general nonzero) expansion.; this is dicsussed in section 4.8.)

Thorn **AHFinderDirect** finds an apparent horizon by numerically solving equation (1). It requires as input the usual Cactus 3-metric g_{ij} and extrinsic curvature K_{ij}, (and optionally the conformal factor ψ if the **StaticConformal** metric semantics are used), and produces as output the Cactus (x, y, z) coordinates of a large number of points on the apparent horizon, together with some auxiliary information like the apparent horizon area and centroid position, and the irreducable mass associated with the area.

Besides this thorn guide, the other main sources of information on **AHFinderDirect** are the comments in the `param.ccl` file, the paper [Tho04], and to a lesser extent the paper [Tho96]. As a courtesy, I ask that both these papers be cited in any published research which uses this thorn, or which uses code from this thorn.

2 What AHFinderDirect Needs

There are some restrictions on the spacetime, or more precisely on each slice where you want to find apparent horizons, which are necessary in order for **AHFinderDirect** to work:

- **AHFinderDirect** requires that the Cactus geometry (g_{ij}, K_{ij}, and optionally ψ) be nonsingular in a neighborhood of the apparent horizon. In particular, this means that it quite certainly will *not* work for spacetimes/slicings which have a singular geometry on the horizon, such as Schwarzschild/Schwarzschild and Kerr/Boyer-Lindquist.[1]

- Less obviously, this also means that if there is a singularity in the geometry somewhere near the apparent horizon, then you need to have a high enough Cactus 3-D grid resolution that the geometry interpolation doesn't "see" the singularity. (If **AHFinderDirect** "sees" the singularity, it may "just" fail to find the horizon, and/or it may report that the interpolated g_{ij} fails to be positive definite or even contains NaNs.)

- At the moment **AHFinderDirect** and the Cactus interpolators don't know how to avoid an excised region, so if the apparent horizon (or any trial horizon surface as the algorithm is iterating towards the apparent horizon) gets too close to an excised region, you'll get garbage results as the interpolator tries to interpolate data from the excised region. I plan to fix this sometime soon.

- **AHFinderDirect** requires that any apparent horizon it's going to (try to) find must be a "Strahlkörper" (literally "ray body", or more commonly "star-shaped region") relative to some local coordinate origin (which you must specify). A Strahlkörper is defined by Minkowski ([Sch86, p. 108]) as

  > a region in n-dimensional Euclidean space containing the origin and whose surface, as seen from the origin, exhibits only one point in any direction.

  In other words, using polar spherical coordinates relative to the local coordinate origin, the apparent horizon's shape must be parameterizable as $r = h(\text{angle})$ for some single-valued function $h : S^2 \to \Re^+$. (**AHFinderDirect** uses precisely this parameterization.)

There are also some restrictions on your Cactus configuration and run; here's what works and what doesn't:

- I *strongly* recommend using a current-CVS checkout of the Cactus flesh and of all relevant thorns. I haven't tested **AHFinderDirect** at all with older versions of the flesh or other thorns.

- **AHFinderDirect** works fine with the **PUGH** unigrid driver and with the **Carpet** mesh-refinement driver. So far as I know it's never been tested with any other driver.

- **AHFinderDirect** works fine in single- or multi-processor Cactus runs.

- Obviously, your Cactus configuration must include **AHFinderDirect**, and your `ActiveThorns` parameter(s) must activate it.

- **AHFinderDirect** inherits from the other thorns (strictly speaking, implementations) listed in table 1, so you'll need them (or more precisely some thorns providing them) in your configuration and activated, too.

- `Grid::domain = "full"`, `"bitant"`, `"quadrant"`, and `"octant"` are supported. Alas, at present rotating (or more precisely nonlocal) symmetry boundary conditions aren't supported.

- The `ADMBase::metric_type` values `"physical"` and `"static conformal"` are supported; for the latter you must have storage turned on for at least the conformal factor `StaticConformal::psi`. (The Cactus 3-D grid functions for 1st and 2nd derivatives of `psi` aren't used.)

[1] Nonsingular slicings of those same spacetimes, such as Schwarzschild/Eddington-Finkelstein, Kerr/Kerr, and Kerr/Kerr-Schild, are fine, and are in fact nice test cases.

Implementation	Typically provided by **Thorn**
Grid	**CactusBase/CartGrid3d**
IO	**CactusBase/IOUtil**
ADMBase	**CactusEinstein/ADMBase**
StaticConformal	**CactusEinstein/StaticConformal**
SpaceMask	**CactusEinstein/SpaceMask**
SphericalSurface	**AEIThorns/SphericalSurface**

Table 1: This table lists all the other implementations from which **AHFinderDirect** inherits, and the thorns which typically provide these implementations.

- **AHFinderDirect** uses the `CCTK_InterpGridArrays()` Cactus global (multi-processor grid array) interpolator API; this is provided by **PUGHInterp** or **CarpetInterp** (so you must have the appropriate one of these thorns compiled in and activated). `CCTK_InterpGridArrays()` in turn uses the new `CCTK_InterpLocalUniform()` processor-local interpolator API; **AHFinderDirect** uses various options in this API which at present are only supported by thorn **AEILocalInterp** (so you must have this thorn compiled in and activated)
- **AHFinderDirect** uses various Cactus reduction APIs to coordinate multi-processor horizon finding, so (even if you're only going to run on a single processor) you must have a reduction thorn like **PUGHReduce** or **CarpetReduce** compiled in and activated.
- At present only a few of **AHFinderDirect**'s parameters are steerable. (This is actually quite easy to fix; I just haven't gotten around to it yet.)
- I think **AHFinderDirect** will "work" with checkpoint/restart, but I haven't tested this yet. Here "work" means the restart will be like starting a new run, in that **AHFinderDirect** will set the initial guess for each horizon in a start-of-a-new-run manner. Alas, it will also write a new `BH_diagnostics` file for each horizon found, overwriting any existing `BH_diagnostics` files. This is a bug, which I plan to fix soon (the right behavior is/will be to *append* to the existing `BH_diagnostics` file).

AHFinderDirect can pass information to the rest of Cactus in several ways; these are described in detail in section 4.7.

3 Obtaining and Compiling AHFinderDirect

You should be able to obtain the source code for this thorn via the usual procedures for anonymous cvs checkout; at present it lives in the **AEIThorns** arrangement.

This thorn is written primarily in C++, calling C and Fortran 77 numerical libraries.[2] In theory the code should be quite portable to modern C++ compilers, but in practice I've had a number of portability problems with various compilers. See the "Code Notes" and "Compiler Notes" sections in the top-level `README` file for details and lists of compilers currently known to be ok or not.

[2]The (C) code for the relativity computations (i.e. the apparent horizon equation itself) is mainly machine-generated from Maple code, which in turn uses a Perl preprocessor. But the machine-generated C code is already in CVS along with the rest of this thorn's source code, so you don't need to have Maple installed unless you want to modify the relativity computations.

By default **AHFinderDirect** doesn't use any external libraries. However, if `HAVE_DENSE_JACOBIAN_LAPACK` is defined in `src/include/config.h`, then **AHFinderDirect** uses the LAPACK library for solving linear equations. In this case you need to configure Cactus with `LAPACK=yes`. See the top-level `README` and `README.library` files for details on this.

4 AHFinderDirect Parameters

This thorn has lots of parameters, but most of them have reasonable default values which you probably won't need to change. Here I describe the parameters which you are likely to want to at least look at, and possibly set explicitly.

Note that all of the "`[`n`]`" parameters are Cactus array parameters, which you need to specify separately in your parameter file for each apparent horizon. **IMPORTANT: Apparent horizons are numbered starting at 1, not 0!** The example in section 8 should make this clear.

4.1 Overall Parameters

`find_every`
: This is an integer parameter specifying how often **AHFinderDirect** should try to find apparent horizons: If `find_every` = 0, **AHFinderDirect** is a no-op. If `find_every` $\neq 0$, **AHFinderDirect** tries to find apparent horizons each `find_every` time steps.[3] The default value is 1, i.e. **AHFinderDirect** tries to find apparent horizons at every time step.

`N_horizons`
: How many apparent horizons do you want to find in each slice? Typical values are 1 (the default), 2, or 3.[4] **This thorn numbers the apparent horizons from 1 to `N_horizons` inclusive.** There are a number of other parameters (described below) which you need to set for of these each apparent horizons.

  Note that `N_horizons` sets the number of apparent horizons you want to find *in the Cactus 3-D numerical grid*, not in the whole spacetime. For example, if you are simulating (say) Misner data with `Grid::domain = "bitant"`, with the two throats at (say) roughly $z = \pm 1$, then you should set `N_horizons = 1` to find those two apparent horizons, since you're only finding one apparent horizon within the numerical grid. If you also want to search for a common apparent horizon surrounding both black holes, then you should set `N_horizons = 2`, since you're finding at most 2 apparent horizons within the numerical grid.

`verbose_level`
: This controls how verbose this thorn is in printing informational (non-error) messages describing what it's doing. In order from tersest to most verbose, the allowable values are

  `"no output"`
  : Don't print anything.

  `"physics highlights"`
  : Print only a single line each time **AHFinderDirect** runs, giving which horizons were found.

[3] More precisely, if `find_every` $\neq 0$, **AHFinderDirect** tries to find apparent horizons at each time step where `cctk_iteration` is evenly divisible by `find_every`.

[4] Larger values are also possible. The present upper limit is 100, but it would be trivial to raise this further if desired – see the comments in `param.ccl` for details.

`"physics details"`
: Print two lines for each horizon found, giving the horizon area, centroid position, and irreducible mass. This is the default.

`"algorithm highlights"`
: Also print a single line for each Newton iteration giving the 2-norm and ∞-norm of the $\Theta(h)$ function defined by equation (1).

`"algorithm details"`
: Print lots of detailed messages tracing what the code is doing.

`"algorithm debug"`
: Print even more detailed messages tracing what the code is doing, mainly useful for debugging purposes.

4.2 Choosing the Local Coordinate Origin for each Apparent Horizon

For each apparent horizon you want to find, you need to specify the Cactus (x, y, z) coordinates of a local coordinate system origin. As described in section 2, each apparent horizon must be a Strahlkörper with respect to its local coordinate system origin.

You specify the local coordinate system origin for each horizon with the (Cactus array) parameters

`origin_x`[n]
`origin_y`[n]
`origin_z`[n]
: These all default to 0.0. In practice, you should set these parameters to be somewhere reasonably close to your best guess for the center of each apparent horizon. These aren't *too* critical: being off by 1/4 of the horizon radius is no problem, and – assuming the algorithm still converges – even 1/2 of the horizon radius only slows the convergence by an extra iteration or two. But poor values of these parameters do make the algorithm more likely to fail to converge.

At present the local coordinate origin is fixed once you set it; there's no provision for it to move to track a moving black hole. I hope to add such a provision soon.[5]

4.3 Specifying the Initial Guess

AHFinderDirect requires an initial guess for the apparent horizon's coordinate position and shape (that is, for the h(angle) function defined in section 2), for each apparent horizon you want to find. Unlike some other apparent horizon finders (eg. the curvature flow method in **AHFinder**), for **AHFinderDirect** there's no restriction on whether the initial guess is inside, outside, or crossing the actual apparent horizon: the only important thing is that it should be "close". Just how close the initial guess needs to be for **AHFinderDirect** to find the (a) apparent horizon depends on the slice and the coordinates, but as a general rule of thumb any initial guess that's within 1/3 to 1/2 of the mean horizon radius will probably work.

The "initial guess" specification is used the first time we try to find any given apparent horizon, and also any succeeding time when the most recent attempt to find this apparent horizon failed. If we succeed in finding a given apparent horizon, than that apparent horizon position is automatically reused as the

[5] This would be along the lines of the **AHFinder** apparent horizon finder's `AHFinder::ahf_wander = true` option.

initial guess the next time we try to find the same apparent horizon; in this case the explicit "initial guess" specification is ignored.[6]

There are a number of parameters for specifying the initial guess:

`initial_guess_method[`n`]`
: This sets what type of the initial guess is used for each apparent horizon position. There are several possibilities, most with their own sets of subparameters:[7,8]

  `"read from file"`
  : This reads the initial-guess h(angle) function from a data file. The file format is currently hard-wired to be that written with `file_format = "ASCII (gnuplot)"` (see below). The subparameter `initial_guess__read_from_named_file__file_name` specifies the file name.

  `"Kerr/Kerr"`
  : This sets the initial guess to the analytically-known apparent horizon position in a Kerr spacetime in Kerr coordinates. This is a coordinate sphere of radius $(1 + \sqrt{1 - a^2})m$. There are subparameters

    `initial_guess__Kerr_Kerr__x_posn[`n`]`
    `initial_guess__Kerr_Kerr__y_posn[`n`]`
    `initial_guess__Kerr_Kerr__z_posn[`n`]`
    : to set the position of the Kerr black hole (note this position is in *global* Cactus coordinates, not relative to the local coordinate origin), and

    `initial_guess__Kerr_Kerr__mass[`n`]`
    `initial_guess__Kerr_Kerr__spin[`n`]`
    : to set its mass and spin.

  `"Kerr/Kerr-Schild"`
  : This sets the initial guess to the analytically-known apparent horizon position in a Kerr spacetime in Kerr-Schild coordinates. This is a coordinate ellipsoid with radia (semi-major axes)

$$\begin{aligned} r_z &= (1 + \sqrt{1 - a^2})m \\ r_x = r_y &= r_z \sqrt{1 + \left(\frac{am}{r_z}\right)^2} = \sqrt{\frac{2r_z}{m}}\, m \end{aligned} \tag{2}$$

    `initial_guess__Kerr_KerrSchild__x_posn[`n`]`
    `initial_guess__Kerr_KerrSchild__y_posn[`n`]`
    `initial_guess__Kerr_KerrSchild__z_posn[`n`]`
    : (note this position is in *global* Cactus coordinates, not relative to the local coordinate origin), and

    `initial_guess__Kerr_KerrSchild__mass[`n`]`
    `initial_guess__Kerr_KerrSchild__spin[`n`]`
    : to set its mass and spin.

  `"coordinate sphere"`
  : This sets the initial guess to a coordinate sphere; there are subparameters

    `initial_guess__coord_sphere__x_center[`n`]`
    `initial_guess__coord_sphere__y_center[`n`]`
    `initial_guess__coord_sphere__z_center[`n`]`

[6] This is similar to the **AHFinder** apparent horizon finder's behavior with the `AHFinder::ahf_guessold = true` option.
[7] The naming scheme for these is similar to that used in the **Exact** thorn.
[8] For the Kerr parameters, note that this thorn always uses the convention that the spin parameter $a \equiv J/m^2$ is dimensionless.

(note this position is in *global* Cactus coordinates, not relative to the local coordinate origin), and

`initial_guess__coord_sphere__radius[`n`]`
to set the radius.

`"coordinate ellipsoid"`
This sets the initial guess to a coordinate ellipsoid; there are subparameters

`initial_guess__coord_ellipsoid__x_center[`n`]`
`initial_guess__coord_ellipsoid__y_center[`n`]`
`initial_guess__coord_ellipsoid__z_center[`n`]`
(note this position is in *global* Cactus coordinates, not relative to the local coordinate origin), and

`initial_guess__coord_ellipsoid__x_radius[`n`]`
`initial_guess__coord_ellipsoid__y_radius[`n`]`
`initial_guess__coord_ellipsoid__z_radius[`n`]`
to set the radia (semimajor axes).

4.4 I/O Parameters for the Apparent Horizon Shape(s)

The main output of this thorn is the computed horizon shape function h(angle), and correspondingly the (x, y, z) coordinate positions of the apparent-horizon-surface (angular) grid points. There are several parameters controlling if, how often, and how these should be written to data files:

`output_h_every`
As described in section 4.1, **AHFinderDirect** will try to find the apparent horizon(s) every `find_every` time steps. However, you can control how often (if at all) the apparent horizon shape(s) are written to data files: this is only done if `output_h_every` is nonzero, and the Cactus time step number `cctk_iteration` is an integral multiple of this parameter (`output_h_every`).

`file_format`
This specifies the file format for horizon-shape (and other angular-grid-function) data files. Unfortunately, at the moment only a single format is implemented,

`"ASCII (gnuplot)"`
This is a simple ASCII format designed for easy plotting with gnuplot:

- **AHFinderDirect** writes a separate data file for each Cactus time step and for each apparent horizon found. By default these are all written in to the `IOUtil::out_dir` directory; see `h_directory` (below) to change this.
- The time step number and the apparent horizon number are both encoded in the file name; the actual file name is given by a `printf()` format `"%s/%s.t%d.ah%d.%s"`, where
  - the first `%s` is the directory set by the `IO::out_dir` and/or `h_directory` parameters (see below)
  - the second `%s` is the base file name set by the `h_base_file_name` parameter (see below)
  - the first `%d` is the Cactus time step number
  - the second `%d` is the apparent horizon number
  - the third `%s` is the file name extension, set by the `ASCII_gnuplot_file_name_extension` parameter; this defaults to `"gp"`
- Comment lines begin with `#`.

- Patches are separated by 2 blank lines; rows of apparent-horizon points within a patch are separated by single blank lines.
- Each apparent-horizon-surface point is described by a single line, containing the whitespace-separated fields
  - Two "unwrapped" angular coordinates in degrees, representing a point on S^2.
  - The h value, giving the radius of the apparent horizon surface at that angle.
  - The corresponding Cactus (x, y, z) coordinates.

See section 6 for a discussion of visualization for these and other **AHFinderDirect** output files.

`h_directory`
: This specifies the directory in which the h data files are to be written. If it doesn't already exist, this directory is created before writing the data files. This parameter defaults to the value of the `IO::out_dir` parameter.

`h_base_file_name`
: This specifies the base file name for h data files, as described above. This defaults to "h".

`ASCII_gnuplot_file_name_extension`
: This specifies the file name extension for h data files, as described above. This defaults to "gp".

4.5 Parameters for the "BH_diagnostics" Files

As well as the apparent horizon shape files, this thorn can also write files giving time series of various diagnostics. These are controlled by the following parameters:

`output_BH_diagnostics`
: If this Boolean parameter is set to true, **AHFinderDirect** will write a "black hole diagnostics" file for each distinct apparent horizon found (up to `N_horizons` files in all). Each such file contains a time series of various diagnostics for all time steps when the corresponding apparent horizon was found. The file format is again a simple ASCII format designed for easy plotting with gnuplot:

  - The apparent horizon number is encoded in the file name; the actual file name is given by a `printf()` format `"%s/%s.ah%d.%s"`, where
    - the first `%s` is the directory set by the `IO::out_dir` and/or `BH_diagnostics_directory` parameters (see below)
    - the second `%s` is the base file name set by the `BH_diagnostics_base_file_name` parameter (see below); this defaults to `"BH_diagnostics"`
    - the `%d` is the apparent horizon number
    - the third `%s` is the file name extension, set by the `BH_diagnostics_file_name_extension` parameter; this defaults to `"gp"`
  - The file begins with a block of header comments (lines begining with `#`) describing the data fields.
  - Each time this apparent horizon is found, a single line is appended to the data file, containing various whitespace-separated fields. The the precise list of fields, see the header comments, or see the function `output()` in `src/driver/BH_diagnostics.cc`. As of this writing the fields are:
    - the Cactus iteration number `cctk_iteration`
    - the Cactus time coordinate `cctk_time`

- the Cactus (x, y, z) coordinates of the apparent horizon centroid
- the minimum, maximum, and mean coordinate radia of the apparent horizon about the local coordinate origin
- the minimum and maximum Cactus x, y, and z coordinates of the apparent horizon surface
- the proper circumferences of the apparent horizon in the xy, xz, and yz local-coordinate planes[9]
- the xz/xy and yz/xy ratios of the proper circumferences
- the proper area of the apparent horizon, A
- the irreducible mass of the apparent horizon, $\sqrt{A/16\pi}$
- the areal radius of the apparent horizon, $\sqrt{A/4\pi}$

`BH_diagnostics_directory`
: This specifies the directory in which the black hole diagnostics data files are to be written. If it doesn't already exist, this directory is created before writing the data files. This parameter defaults to the value of the `IO::out_dir` parameter.

`BH_diagnostics_base_file_name`
: This specifies the base file name for black hole diagnostics data files, as described above. This defaults to `"BH_diagnostics"`.

`BH_diagnostics_file_name_extension`
: This specifies the file name extension for black hole diagnostics data files, as described above. This defaults to `"gp"`.

4.6 (Excision) Mask Parameters

This thorn can optionally set a mask grid function (or functions) at each point of the Cactus grid, to indicate where that point is with respect to the apparent horizon(s). This is usually used for excision.

`set_mask_for_all_horizons`
`set_mask_for_individual_horizon[`n`]`
: These Boolean parameters control whether **AHFinderDirect** should set a mask grid function(s): If the (C) expression `set_mask_for_all_horizons || set_mask_for_individual_horizon[i]` is true, then **AHFinderDirect** will set a mask grid function(s) for apparent horizon i. All these parameters default to false (don't set a mask).

  If any of these parameters is set to true, you almost certainly also need to set the Boolean parameter `SpaceMask::use_mask` to true to to turn on storage for the mask grid function(s)!

If it's setting a mask(s), **AHFinderDirect** partitions the Cactus grid into 3 regions: an "inside", a "buffer", and an "outside". Typically the inner region is excised, but **AHFinderDirect** doesn't itself do this: It just sets the mask(s); you need to use some other thorn(s) to do the actual excision.

The 3 regions are defined as follows: For a grid point a distance $r[i]$ from horizon i's local coordinate origin, with horizon i's radius in this same direction (again, measured from its local coordinate origin)

[9] Note that if the apparent horizon is not symmetric across one of these coordinate planes, then the circumference in this plane is not very meaningful. In particular, in this case this circumference is probably different from the apparent horizon's maximum "hoop" circumference.

being $r_{\rm horizon}[i]$,[10] the regions are defined by

$$\begin{array}{lllll} r[i] \le r_{\rm inner}[i] \text{ for some } i & & & \Rightarrow & \text{inner} \\ r[i] > r_{\rm inner}[i] \text{ for all } i & \text{and} & r[i] \le r_{\rm outer}[i] \text{ for some } i & \Rightarrow & \text{buffer} \\ r[i] > r_{\rm outer}[i] \text{ for all } i & & & \Rightarrow & \text{outer} \end{array} \tag{3}$$

where

$$\begin{aligned} r_{\rm inner} &= \texttt{mask\_radius\_multiplier} \times r_{\rm horizon} + \texttt{mask\_radius\_offset} \times \Delta x_{\rm base} \\ r_{\rm outer} &= r_{\rm inner} + \texttt{mask\_buffer\_thickness} \times \Delta x_{\rm base} \end{aligned} \tag{4}$$

and where $\Delta x_{\rm base}$ is the base-grid Cactus grid spacing (more precisely, the geometric mean of the base grid's x, y, and z Cactus grid spacings)[11].

`mask_radius_multiplier`
`mask_radius_offset`
`mask_buffer_thickness`

These parameters are used to define the radia $r_{\rm inner}$ and $r_{\rm outer}$ in equation (4) above.

Note that the sign convention here is that `mask_radius_multiplier` is *multiplied* by the horizon radius, then `mask_radius_offset` (scaled by the Cactus grid spacing) is *added*. Thus for use with excision (where the inner region – which will be excised – must be somewhat *inside* the horizon), `mask_radius_multiplier` should be a positive real number slightly less than 1.0, and/or `mask_radius_offset` a negative real number.

The default values for these parameters are

```
mask_radius_multiplier =  0.8
mask_radius_offset     = -5.0
mask_buffer_thickness  =  5.0
```

`mask_is_noshrink`

This Boolean parameter specifies whether the inside and buffer regions should be prevented from ever shrinking during a time evolution (this is the default), or whether they should be set independently from one time step to the next (and thus allowed to either grow or shrink). More precisely, once a given grid point has been classified as inside, buffer, or outside, **AHFinderDirect** executes the following algorithm:

inside

`mask` ← `inside_value`

buffer

```
if (mask_is_noshrink && (mask = inside_value))
  then {
       # this point was previously inside ⇒ no-op here
       }
  else mask ← buffer_value
```

outside

[10] Note that these are flat-space distances $[(x-x_0)^2+(y-y_0)^2+(z-z_0)^2]^{1/2}$ – the spacetime metric is *not* used here.

[11] The "base grid" part here only matters if you're doing mesh refinement, in which case it's necessary to keep excision consistent between the different refinement levels.

```
if (mask_is_noshrink && ((mask = inside_value) || (mask = buffer_value))
  then {
       # this point was previously inside or buffer ⇒ no-op here
       }
  else  mask ← outside_value
```

`min_horizon_radius_points_for_mask`
: By default, **AHFinderDirect** sets the mask for each apparent horizon found. If we're using mesh refinement, it's possible for an apparent horizon to be found on a coarse grid, and the masked region to be only a few grid points across on a fine grid. This causes some other Cactus thorns (eg. **LegoExcision**) to crash. :(

  This parameter can be used to avoid this problem: For each apparent horizon that it finds, **AHFinderDirect** only sets the mask on a given grid if

$$r_{\text{inner,min}} \geq \texttt{min\_horizon\_radius\_points\_for\_mask} * \Delta x_{\text{current,max}} \tag{5}$$

  where $r_{\text{inner,min}}$ is the minimum over all angles of r_{inner} as defined by equation (4), and $\Delta x_{\text{current,max}}$ is the maximum of the Cactus x, y, and z grid spacings on the current Cactus grid.[12] If this condition isn't satisfied, then **AHFinderDirect** skips setting the mask for this apparent horizon, just as if this apparent horizon wasn't found. (Note that all other processing for an apparent horizon being found is still done, including writing output files, using the apparent horizon shape as the initial guess for the next time step's apparent-horizon finding, etc.; it's only the mask processing for this horizon (at this time step) that's skipped.)

AHFinderDirect supports two types of mask grid functions; the following two Boolean parameters choose which of them you want to set; you can set either or even both of these:

`set_old_style_mask`
: This parameter (default `true`) specifies an old-style excision mask, one stored in a `CCTK_REAL` Cactus grid function. (The **AHFinder** apparent horizon finder uses this type of mask.)

`set_new_style_mask`
: This parameter (default `false`) specifies a new-style excision mask, one stored in a specified bit field of a `CCTK_INT` Cactus grid function. The bit field is specified by its name, as registered with the **SpaceMask** thorn. We plan to eventually convert all Cactus excision (and other uses of mask grid functions) to this scheme, but at the moment not much code supports it. Note that **AHFinderDirect** doesn't itself create/register any bit fields or state names with **SpaceMask** – you must arrange for some other thorn(s) to do this.

For an old-style mask, the following parameters specify the mask grid function and how it should be set:

`old_style_mask_gridfn_name`
: This parameter specifies the mask grid function's name.

`old_style_mask_inside_value`
`old_style_mask_buffer_value`
`old_style_mask_outside_value`
: If an old-style mask is to be set in the corresponding regions, these parameters specify the values to which it should be set. These are all `CCTK_REAL` values.

[12] Note that this is the **current** Cactus grid spacing, not the **base** Cactus grid spacing that's used when calculating r_{outer} and r_{inner} by equation (4).

For an new-style mask, the following parameters specify the mask grid function and how it should be set:

`new_style_mask_gridfn_name`
`new_style_mask_bitfield_name`
: These parameters specify the mask grid function's name and the bitfield name within it.

`new_style_mask_inside_value`
`new_style_mask_buffer_value`
`new_style_mask_outside_value`
: If an new-style mask is to be set in the corresponding regions, these parameters specify the values to which it should be set. These are all character-string state names, as registered with the **SpaceMask** thorn.

Note that **AHFinderDirect** doesn't itself register any bitfields or states with **SpaceMask** – you must arrange for some other thorn(s) to do this before **AHFinderDirect** tries to find the horizon(s).

If **AHFinderDirect** sets a mask or masks, this happens in the same schedule bin(s) as the horizon finding. More precisely, **AHFinderDirect** creates two schedule groups for this purpose:

- The schedule group `group_for_mask_stuff` is scheduled to run just after the horizon is found.
- The schedule group `group_where_mask_is_set` is scheduled inside the schedule group `group_for_mask_stuff`.
- The actual setting of the mask is scheduled inside the schedule group `group_where_mask_is_set`.

Thorn **PreviousMask** uses these schedule groups to keep a "previous" as well as a "current" mask. See that thorn's thorn guide for further details.

4.7 Communicating with Other Thorns

Besides the data files it writes, **AHFinderDirect** currently has three ways to communicate with other Cactus thorns:

- **AHFinderDirect** can set a mask grid function(s) based on the (a) horizon's shape; other thorns may then use this for excision or other purposes. **AHFinderDirect** supports both the old-style (`CCTK_REAL`) mask (compatible with **AHFinder**) or the new-style (`CCTK_INT`) mask bit-fields defined by **SpaceMask**; you can even use both styles simultaneously.
- **AHFinderDirect** can announce a selected horizon's centroid position to another thorn (typically **DriftCorrect**, which uses this to adjust its corotating shift vector). This uses the new function-aliasing version of **DriftCorrect**, not the old version which worked with an auxiliary thorn **AHFSetDCCentroid**.
- **AHFinderDirect** provides a set of aliased functions which any other thorn(s) can call to find out the shape of a specified horizon.
- **AHFinderDirect** can store information about the horizon(s) it finds in the **SphericalSurface** variables for other thorns to use.

AHFinderDirect's mask features are described in section 4.6; the other communication mechanisms are described in the following subsections.

4.7.1 Parameters for Announcing a Horizon Centroid to Other Thorns

This thorn can optionally announce the centroid of a specified apparent horizon to another thorn (typically **DriftCorrect**) each time that apparent horizon is found. This is controlled by the following parameter:

`which_horizon_to_announce_centroid`
: This is an integer parameter which defaults to 0 (which means not to announce any centroid). If it's set to a nonzero integer, that specifies the horizon number to have its centroid announced.

4.7.2 Aliased Functions to Provide Horizon-Shape Information

AHFinderDirect provides the following aliased functions to allow other thorns to find out about the horizons. Each function returns a status code which is ≥ 0 for ok, or negative for an error.

```
#
# This function returns the local coordinate origin for a given horizon.
#
CCTK_INT FUNCTION HorizonLocalCoordinateOrigin                          \
   (CCTK_INT IN horizon_number,                                         \
    CCTK_REAL OUT origin_x, CCTK_REAL OUT origin_y, CCTK_REAL OUT origin_z)

#
# The following function queries whether or not the specified horizon
# was found the most recent time AHFinderDirect searched for it.
# The return value is:
#  1 if the horizon was found
#  0 if the horizon was not found
#  negative for an error
#
CCTK_INT FUNCTION HorizonWasFound(CCTK_INT IN horizon_number)

#
# The following function computes the horizon radius in the direction
# of each (x,y,z) point, or -1.0 if this horizon wasn't found the most
# recent time AHFinderDirect searched for it.  More precisely, for each
# (x,y,z), consider the ray from the local coordinate origin through
# (x,y,z).  This function computes the Euclidean distance between the
# local coordinate origin and this ray's intersection with the horizon,
# or -1.0 if this horizon wasn't found the most recent time AHFinderDirect
# searched for it.
#
# If this function is to be used in a multiprocessor run on a horizon
# which was found on some other processor, the parameter
#   AHFinderDirect::always_broadcast_horizon_shape
# must be set to true to get the correct answer.
#
CCTK_INT FUNCTION HorizonRadiusInDirection                              \
   (CCTK_INT IN horizon_number,                                         \
    CCTK_INT IN N_points,                                               \
```

```
CCTK_REAL IN ARRAY x, CCTK_REAL IN ARRAY y, CCTK_REAL IN ARRAY z,   \
CCTK_REAL OUT ARRAY radius)
```

4.7.3 Storing Horizon-Shape Information in the SphericalSurface Variables

SphericalSurface (in the **AEIThorns** arrangement) defines a set of generic grid arrays which describe "spherical surfaces". **AHFinderDirect** can optionally store information about the horizons it finds in the **SphericalSurface** variables. This is controlled by the following parameters:

`which_surface_to_store_info[`n`]`
: This parameter should be set to the **SphericalSurface** surface number into which information on a given **AHFinderDirect** horizon should be stored, or to -1 to skip storing the information. It defaults to -1 for each horizon.

  Note that SphericalSurface numbers surfaces starting from 0, whereas AHFinderDirect numbers horizons starting from 1!

  At present, if multiple **AHFinderDirect** horizons specify the same **SphericalSurface** surface, the highest-numbered horizon will "win", i.e. it will overwrite the data from any lower-numbered horizons.

4.8 Other Parameters

`run_at_CCTK_ANALYSIS`
`run_at_CCTK_POSTSTEP`
`run_at_CCTK_POSTINITIAL`
`run_at_CCTK_POST_RECOVER_VARIABLES`
: These parameters (which default to `true`, `false`, `false`, and `true` respectively) control which schedule bins **AHFinderDirect** runs in. Historically, **AHFinderDirect** ran in CCTK_ANALYSIS, and that's still the default, but these parameters allow you to change this so it runs in CCTK_POSTSTEP and/or CCTK_POSTINITIAL instead. (You can even run in all three bins if you want!)

  In general we need to run at CCTK_POST_RECOVER_VARIABLES, since

  - parameters may have been steered at recovery, so we may need to find a new horizon or horizons, and
  - we need to set the mask again to make sure it's correct right away (since our next regular horizon-finding may not be until some time steps later)

  Therefore the `run_at_CCTK_POST_RECOVER_VARIABLES` parameter should probably be left at its default setting of `true`.

`geometry_interpolator_name`
`geometry_interpolator_pars`
: These parameters control the (3-D) "geometry interpolation" of the spacetime geometry (g_{ij} and K_{ij}, or their **StaticConformal** equivalents) to the apparent horizon position. The defaults are set to use a quadratic Hermite interpolator. This works fairly well, but because of the interpolator molecule size you must use `driver::ghost_size` ≥ 2.

  If you want to get very high accuracy from **AHFinderDirect**, then you should use a cubic Hermite geometry interpolator, by setting

```
AHFinderDirect::geometry_interpolator_pars = "
  order=3
  boundary_off_centering_tolerance={1.0e-10 1.0e-10 1.0e-10 1.0e-10 1.0e-10 1.0e-10}
  boundary_extrapolation_tolerance={0.0 0.0 0.0 0.0 0.0 0.0}
                                                "
```

Assuming perfectly accurate geometry variables in the 3-D Cactus grid, this will make **AHFinderDirect** (very) roughly an order of magnitude more accurate. However, the larger molecule size will make it about a factor of 2–3 slower, and will also require that you set `driver::ghost_size` ≥ 3. The sample parameter file `par/Kerr-order3.par` shows an example of this.

`N_zones_per_right_angle[`n`]`

This parameter sets the angular resolution used to compute each patch. The units are the number of angular grid zones per right angle. The default is 18, i.e. a 5 degree angular resolution. There's no problem with this parameter varying from one horizon to another, but for simplicity it should be even.[13]

For any horizon which is close to spherical about its local coordinate origin, you can lower this parameter to make **AHFinderDirect** run faster (typical run-times scale roughly as the cube of this parameter); 6 is about the minimum reasonable value.

For any horizon which is highly non-spherical about its local coordinate origin, you can raise this parameter to get better resolution; 30 should be enough for even a highly non-spherical horizon.

`max_allowable_horizon_radius[`n`]`

This parameter gives the maximum mean-coordinate-radius which any given trial surface may have in the course of trying to solve the apparent horizon equation.[14] In particular, if any trial surface has a mean coordinate radius which exceeds this parameter, **AHFinderDirect** gives up and deems this apparent horizon to be "not found".

This parameter defaults to 10^{10} (effectively $+\infty$) for each apparent horizon. You can set it to a smaller value to make **AHFinderDirect** a bit more efficient, or (probably more important in practice) to stop **AHFinderDirect** from iterating off the edge of the grid if this causes problems with interpolation or boundary conditions.

`max_allowable_Theta`

This parameter gives the maximum $\|\Theta\|_\infty$ which any given trial surface may have in the course of trying to solve the apparent horizon equation. In particular, if any trial surface has $\|\Theta\|_\infty$ exceeding this parameter, **AHFinderDirect** gives up and deems this apparent horizon to be "not found". This parameter defaults to 10^{10} (effectively $+\infty$) for each apparent horizon.

`surface_expansion[`n`]`

This parameter (which defaults to 0.0) sets the expansion of each surface. With the default setting this thorn solves (1) to find apparent horizons. With other settings of this parameter this thorn can be used to find "surfaces of constant expansion"; these may be useful for excision, wave extraction, or other purposes ([Sch03]).

To help in choosing the value(s) of the `surface_expansion[`n`]` parameter, figure 1 (from [Tho96]), shows the expansion of r = constant surfaces in an Eddington-Finkelsteon slice of the unit-mass Schwarzschild spacetime.

[13] More precisely, it *must* be even for any horizon whose patch system type is anything other than `"full sphere"`. See the comments in `param.ccl` for further information.

[14] Note that this is an unweighted arithmetic mean over the horizon-surface grid points, the same value which is printed as `r_grid` during the horizon-finding iterations.

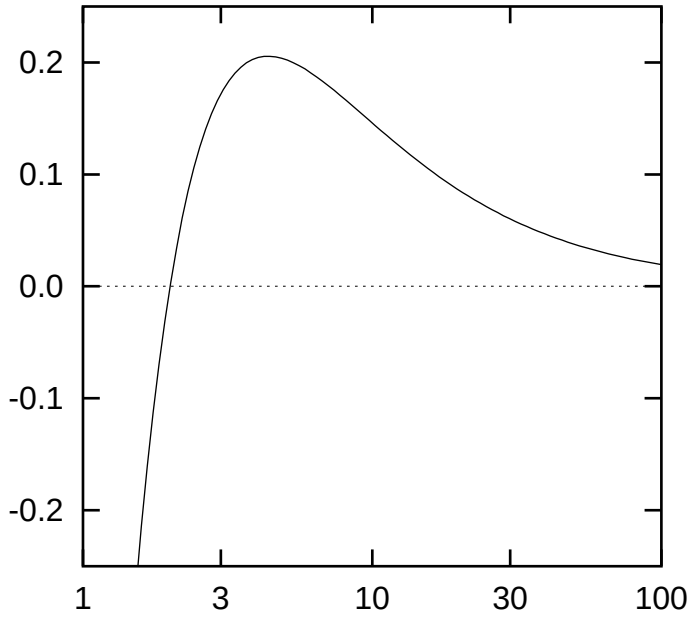

Figure 1: This figure shows the expansion Θ for r = constant surfaces in an Eddington-Finkelstein slice of the unit-Mass Schwarzschild Spacetime.

5 Monitoring AHFinderDirect's Status

There are two primary ways of monitoring what **AHFinderDirect** is doing during a Cactus run: the `BH_diagnostics` files and the `CCTK_INFO` messages written to the Cactus standard output:

The `BH_diagnostics` files are described in detail in section 4.5. These files are written and "flushed" at each time step, so they're always up-to-date.

During the apparent-horizon–finding process, **AHFinderDirect** writes various `CCTK_INFO` messages describing the convergence of the iterative solution of the apparent horizon equation 1 on each processor. In particular, if `verbose_level` is set to `"algorithm highlights"` or a more verbose setting (cf. section 4.1), then **AHFinderDirect** writes `CCTK_INFO` messages like these:

```
INFO (AHFinderDirect):    proc 0/horizon 1:it 1 r_grid=0.595 ||Theta||=1.1e-01
INFO (AHFinderDirect):    proc 0/horizon 1:it 2 r_grid=0.614 ||Theta||=7.2e-02
INFO (AHFinderDirect):    proc 0/horizon 1:it 3 r_grid=0.632 ||Theta||=2.9e-02
INFO (AHFinderDirect):    proc 0/horizon 1:it 4 r_grid=0.642 ||Theta||=9.9e-04
INFO (AHFinderDirect):    proc 0/horizon 1:it 5 r_grid=0.642 ||Theta||=7.9e-07
INFO (AHFinderDirect):    proc 0/horizon 1:it 6 r_grid=0.642 ||Theta||=7.2e-13
INFO (AHFinderDirect): AH 1/2: r=0.660716 at (0.000000,0.000000,1.127434)
INFO (AHFinderDirect): AH 1/2: area=338.0473838 m_irreducible=2.59330658
INFO (AHFinderDirect): writing h to "misner.h.t0.ah1.gp"
```

Here `r_grid` is a rough estimate of the mean radius of the trial surface at each iteration, and `||Theta||` is the infinity-norm of Θ, the left hand side of the apparent horizon equation 1 over the surface. Once the apparent horizon has been found (`||Theta||` is sufficiently small), then **AHFinderDirect** prints its

mean radius,[15] centroid position, area, and irreducible mass.

6 Visualization

There are several ways to visualize **AHFinderDirect**'s output:

The simplest is to plot various quantities from the `BH_diagnostics` files (described in detail in section 4.5). For example, using gnuplot (`http://www.gnuplot.info`), you can plot a graph of the surface area of horizon #4 as a function of coordinate time, with the command

```
plot 'BH_diagnostics.ah4.gp' using 2:26 with points
```

ygraph (`http://www.aei.mpg.de/~pollney/ygraph/`) may also be able to directly plot the `BH_diagnostics` files.

Given a horizon-shape data file `h.t105.h4.gp`, the gnuplot command

```
splot 'h.t105.h4.gp' with lines
```

will plot the h(angle) function, with the x and y axes of the plot being the two "unwrapped" angular coordinates on S^2, in degrees, and the z axis being h(angle). However, in practice this usually isn't very informative. Instead, you probably want the gnuplot command

```
splot 'h.t105.h4.gp' using 4:5:6 with lines
```

which will plot the 3-D shape of the apparent horizon surface.

The `src/misc` directory of the **AHFinderDirect** source code contains several perl scripts which are useful in visualizing **AHFinderDirect** output. In particular, the script `select.plane` selects a particular 2-D plane. If you put this in your Unix path, it can be used with a gnuplot command like

```
splot '<select.plane xy <h.t105.h4.gp' using 4:5 with lines
```

to show the shape of an apparent horizon in the xy plane.[16]

Another Visualization option is OpenDX (`http://www.opendx.org/`). Thomas Radke's has written some OpenDX macros **ImportAHFinderDirectGnuplot.net** and **ImportAHFinderDirectGnuplotPatch.net** to import **AHFinderDirect** horizon-shape data files. These macros use a set of "control files" named `*.dx`, one per horizon, which **AHFinderDirect** (by default) writes into the same directory as the main horizon–shape output files. You can get these macros by anonymous CVS with the command

```
cvs -d :pserver:cvs_anon@cvs.aei.mpg.de:/numrelcvs \
     checkout AEIPhysics/Visualization/OpenDX
```

[15] Note that this may differ significantly from the last iterations' estimated radius. This is because the mean radius printed during the iterations is only a rough estimate (it's the unweighted arithmetic mean of the radius at all the horizon-surface grid points), while the radius printed after the horizon is found is a more accurate value (it's computed via numerical integrals over the surface, taking into account the induced metric).

[16] Note that this script isn't very smart: it doesn't know about the **AHFinderDirect** local coordinate origin. That is, for example, if you select the xy plane, you're selecting the *global* xy plane ($z = 0$).

7 Accuracy

The apparent horizon positions are typically computed very accurately; tests on Kerr spacetimes give typical errors of $10^{-4}m$ to $10^{-5}m$.

The various diagnostics printed to standard output and written to the black hole diagnostics file(s), are typically computed to accuracies on the order of a part per million or so.

Note, however, that the irreducible mass $m_{\text{irreducible}}$ may differ considerably from the black hole's local mass or its contribution to the slice's ADM mass. For example, for Kerr spacetime in Kerr-Schild coordinates, $m_{\text{irreducible}}/m_{\text{ADM}} = 0.949$, 0.894, and 0.723 for spin parameters $a \equiv J/m^2 = 0.6$, 0.8, and 0.999, respectively. It would be better to (also) use the "isolated horizons" formalism of [DKSS03]; at some point this thorn may be enhanced to do this.

8 Examples

There are a few example parameter files in the `par/` directory, including Kerr initial data, Misner initial data, and Misner time-evolution tests. The `Kerr-tiny.par` parameter file is close to a minimal **AHFinderDirect** example:

```
# This parameter file sets up Kerr/Kerr-Schild initial data, then
# finds the apparent horizon in it.  The local coordinate system origin
# and the initial guess are both deliberately de-centered with respect
# to the black hole, to make this a non-trivial test for the apparent
# horizon finder.
#
# This parameter file is "tiny" in the sense that it sets only a
# small number of AHFinderDirect parameters.

# flesh
cactus::cctk_itlast = 0

ActiveThorns = "PUGH"
driver::ghost_size = 2
driver::global_nx = 31
driver::global_ny = 31
driver::global_nz = 19

ActiveThorns = "CoordBase CartGrid3D"
grid::domain = "bitant"
grid::avoid_origin = false
grid::type = "byspacing"
grid::dxyz = 0.2

ActiveThorns = "ADMBase ADMCoupling StaticConformal Spacemask CoordGauge Exact"
ADMBase::initial_lapse = "exact"
ADMBase::initial_shift = "exact"
ADMBase::initial_data = "exact"
ADMBase::lapse_evolution_method = "static"
```

```
ADMBase::shift_evolution_method = "static"
ADMBase::metric_type = "physical"
Exact::exact_model = "Kerr/Kerr-Schild"
Exact::Kerr_KerrSchild__mass = 1.0
Exact::Kerr_KerrSchild__spin = 0.6

########################################

ActiveThorns = "IOUtil"
IOUtil::parfile_write = "no"

########################################

ActiveThorns = "SphericalSurface"
ActiveThorns = "AEILocalInterp PUGHInterp PUGHReduce AHFinderDirect"
AHFinderDirect::h_base_file_name     = "Kerr-tiny.h"

AHFinderDirect::N_horizons = 1
AHFinderDirect::origin_x[1] = 0.5
AHFinderDirect::origin_y[1] = 0.7
AHFinderDirect::origin_z[1] = 0.0

AHFinderDirect::initial_guess_method[1] = "coordinate sphere"
AHFinderDirect::initial_guess__coord_sphere__x_center[1] = -0.2
AHFinderDirect::initial_guess__coord_sphere__y_center[1] =  0.3
AHFinderDirect::initial_guess__coord_sphere__z_center[1] =  0.0
AHFinderDirect::initial_guess__coord_sphere__radius[1] = 2.0
```

9 Surfaces of Constant Expansion

Surfaces of Constant Expansion (CE surfaces) are introduced in [Sch03] as a generalisation of apparent horizons (AH). On an AH surface, the expansion is zero everywhere. On a CE surfaces, the expansion is still everywhere the same, but it need not be zero. CE surfaces are also a generalisation of Constant Mean Curvature surfaces (CMC surfaces); both are identical when the extrinsic curvature vanishes. As described in [Sch03], it is likely that CE surfaces foliate the spacelike hypersurface outside of some interior region. This interior region is inside the common apparent horizon, if it exists.

CE surfaces can give some insight into the spacetime, because they can be used to analyse the part of the spacelike hypersurface "between the horizons and infinity". Most notably, they can be used to look at the region where a common horizon is about to (or believed to) form. Similarly, one can use them for collapsing stars where an apparent horizon has not yet formed.

10 Pretracking

Apparent horizon pretracking is introduced in [Sch03]. This is an application of CE surfaces. Even when there is no common horizon, there are still common CE surfaces surrounding multiple black holes. Pretracking consists of tracking in time the smallest common CE surface that can be found. It is reasonable to believe that this surface will evolve into the common horizon at the time where this common

horizon begins to exist. The expansion of this smallest CE surface is also an indication of how close the spacelike hypersurface is to having a common apparent horizon.

11 How AHFinderDirect Works

AHFinderDirect uses the apparent horizon (henceforth "horizon") finding algorithm of [Tho96], modified slightly to work with g_{ij} and K_{ij} on a Cartesian (xyz) grid. The algorithm is described in detail in [Tho04].

11.1 General Description of the Algorithm

As described above, I parameterizes the horizon shape by $r = h(\text{angle})$ for some single-value function $h : S^2 \to \Re^+$. The apparent horizon equation (1) then becomes a 2-D elliptic PDE on S^2 for the function h. I finite difference this in angle to obtain a system of simultaneous nonlinear algebraic equations for h at the angular grid points, and solve this system of equations by a global Newton's method (or a variant with improved convergence).

Computationally, this algorithm has 3 main parts:

- Computation of the "horizon function" $\Theta(h)$ given a trial surface defined by a trial horizon shape function h. This is done by interpolating the Cactus geometry fields g_{ij} and K_{ij} (and optionally ψ) from the 3-D xyz grid to the (2-D set of) trial-horizon-surface grid points (also computing $\partial_k g_{ij}$ in the interpolation process), then doing all further computations with angular grid functions defined solely on S^2 (i.e. at the horizon-surface grid points).
- Computation of the Jacobian matrix $\mathbf{J}\big[\Theta(h)\big]$ of $\Theta(h)$. This thorn incorporates the "symbolic differentiation" technique described in [Tho96], so this computation is quite fast. The Jacobian is a highly sparse matrix; **AHFinderDirect** has code to store it as either a dense matrix (for debugging purposes), or a sparse matrix (the default). Which option is used is determined by a compile-time configuration in `src/include/config.h`.
- Solving the nonlinear equations $\Theta(h) = 0$ by a global Newton's method or a variant. How this is done depends on how the Jacobian is stored. At present,
  - If **AHFinderDirect** is configured to store the Jacobian as a dense matrix, then LAPACK is used to solve the linear equations.
  - If **AHFinderDirect** is configured to store the Jacobian as a sparse matrix, then an incomplete-LU-decomposition–conjugate-gradient solver is used.

  By default only the sparse-matrix code is configured, so LAPACK isn't used and there's no need to link with the LAPACK library.

11.2 The Multipatch System

Perhaps the most unusual feature of **AHFinderDirect** is the "multipatch" system used to cover S^2 without coordinate singularities. In general there are 6 patches, one each covering a neighborhood of the $\pm z$, $\pm x$, and $\pm y$ axes, but this may be reduced in the presence of suitable symmetries. For example, figure 2 on page 22 shows a system of 3 patches covering the $+xyz$ octant of S^2. This would be suitable

for finding an apparent horizon with mirror symmetry about the (local) $z = 0$ plane, and either 90 degree periodic rotation symmetry about the (local) z axis, or mirror symmetry about each of the (local) x and y axes.

To allow easy angular finite differencing within the patch system, each patch is extended beyond its nominal extent by a "ghost zone"[17] (2 grid points wide in figure 2). Angular grid function values in the ghost zone can be obtained by interpatch interpolation[18] or by applying symmetry operations. Once this is done, then angular finite differencing within the nominal extent of each patch can proceed normally, ignoring the patch boundaries. **AHFinderDirect** can be configured at compile time to use either 2nd order or 4th order angular finite differencing (3 point or 5 point angular molecules); the default is 4th order (5 point). This is configured at compile time in `src/include/config.h`.

By default **AHFinderDirect** will automagically choose a patch system type for each apparent horizon searched for, based on the local coordinate origin and the symmetries implicit in the Cactus grid type. This generally works well, but if desired you can instead manually specify the patch system type, the angular resolution, the width of the ghost zones, etc. See the `param.ccl` file for details.

11.3 Other Software Used

AHFinderDirect's `src/sparse-matrix/` directory contains various sparse-matrix libraries, which have their own copyrights and licensing terms:

The `src/sparse-matrix/umfpack/` directory contains a subset of the files in UMFPACK version 4.0 (11.Apr.2002). This code is copyright (©) 2002 by Timothy A. Davis, and is subect to the UMFPACK License:

```
Your use or distribution of UMFPACK or any modified version of
UMFPACK implies that you agree to this License.

THIS MATERIAL IS PROVIDED AS IS, WITH ABSOLUTELY NO WARRANTY
EXPRESSED OR IMPLIED.  ANY USE IS AT YOUR OWN RISK.

Permission is hereby granted to use or copy this program, provided
that the Copyright, this License, and the Availability of the original
version is retained on all copies.  User documentation of any code that
uses UMFPACK or any modified version of UMFPACK code must cite the
Copyright, this License, the Availability note, and "Used by permission."
Permission to modify the code and to distribute modified code is granted,
provided the Copyright, this License, and the Availability note are
retained, and a notice that the code was modified is included.  This
software was developed with support from the National Science Foundation,
and is provided to you free of charge.
```

[17] Note that this terminology differs somewhat from that used by Cactus in general; Cactus would call these "patch zones" or "symmetry zones".

[18] Due to the way the patch coordinates are defined, adjacent patches always share a common "perpendicular" angular coordinate, so only 1-D interpolation is needed here.

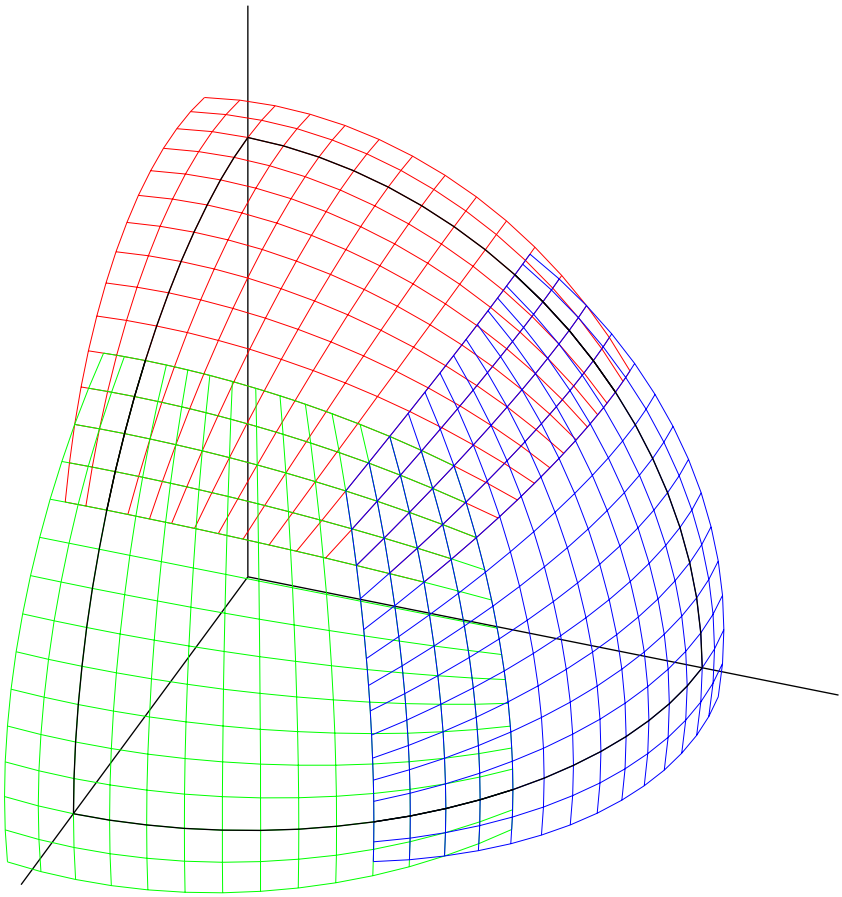

Figure 2: This figure shows a multipatch system covering the $(+,+,+)$ octant of the unit sphere S^2 with 3 patches. The angular resolution is 5 degrees. Notice that the patches overlap by several "ghost zone" grid points.

12 Other Related Thorns

If you're interested in **AHFinderDirect**, you might also be interested in some other related thorns:

EHFinder (in the **AEIDevelopment** arrangement) was written by Peter Diener, and finds the *event* horizon(s) in a numerically computed spacetime. It's described in detail in the paper [Die03].

AHFinder (in the **CactusEinstein** arrangement) was written by Miguel Alcubierre, and includes two different algorithms for finding apparent horizons, a minimization method and a "fast flow" method based on [Gun98]. Unfortunately, both methods are very slow in practice.

TGRapparentHorizon2D (in the **TAT** arrangement) was written by Erik Schnetter, and is another apparent horizon finder. It uses methods very similar to this thorn, and (like this thorn) is very fast and accurate. However, it's no longer under active development. It's described in detail in the papers [Sch02] and [Sch03].

AHFinderDirect (Erik branch) (in the **AEIThorns** arrangement)
Erik Schnetter has added a number of new features to **AHFinderDirect** on a CVS branch with the tag **Erik**, including horizon pretracking (to locate places where horizons are about to form), and the ability to find constant-expansion and constant-mean-curvature surfaces specified by their areal radius. We hope to integrate these into the main **AHFinderDirect** branch during the summer of 2004.

13 Acknowledgments

I thank Peter Diener, Ian Hawke, and Erik Schnetter for many valuable conversations. I think Thomas Radke for his work on the new interpolators. I thank the whole Cactus crew for a great infrastructure!

Erik Schnetter originally implemented a number of improvements to this thorn, notably the **SphericalSurface** interface and the new features in the **Erik** branch.

I thank the Alexander von Humboldt foundation and the AEI visitors' and postdoctoral fellowships programs for financial support.

References

[Die03] P. Diener. A new general purpose event horizon finder for 3D numerical spacetimes. *Class. Quantum Grav.*, 20(22):4901–4917, 2003.

[DKSS03] Olaf Dreyer, Badri Krishnan, Deirdre Shoemaker, and Erik Schnetter. Introduction to Isolated Horizons in Numerical Relativity. *Phys. Rev. D*, 67:024018, 2003.

[Gun98] Carsten Gundlach. Pseudo-spectral apparent horizon finders: An efficient new algorithm. *Phys. Rev. D*, 57:863–875, 1998.

[Sch86] Manfred R. Schroeder. *Number Theory in Science and Communication.* Springer-Verlag, Berlin, 2nd enlarged edition, 1986.

[Sch02] Erik Schnetter. A fast apparent horizon algorithm. gr-qc/0206003, 2002.

[Sch03] Erik Schnetter. Finding apparent horizons and other two-surfaces of constant expansion. *Class. Quantum Grav.*, 20(22):4719–4737, 2003.

[Tho96] Jonathan Thornburg. Finding apparent horizons in numerical relativity. *Phys. Rev. D*, 54(8):4899–4918, October 15 1996.

[Tho04] Jonathan Thornburg. A fast apparent-horizon finder for 3-dimensional Cartesian grids in numerical relativity. *Class. Quantum Grav.*, 21(2):743–766, 21 January 2004.

[Yor89] James W. York. Initial data for collisions of black holes and other gravitational miscellany. In C. Evans, L. Finn, and D. Hobill, editors, *Frontiers in Numerical Relativity*, pages 89–109. Cambridge University Press, Cambridge, England, 1989.